

CS6650 Midterm Mastery

Step III: Crashing and Recovering

Cascading Failure Demo with Resilience Patterns

ECS Fargate | Terraform | Go | Circuit Breaker | Bulkhead | Fail Fast

1. Overview

This report documents a crash-and-recovery experiment built on top of the Homework 6 product search service. The goal was to demonstrate how cascading failures propagate through a distributed system when a downstream dependency degrades, and how resilience patterns from Sam Newman's Building Microservices can protect the system from total failure.

The experiment was deployed to AWS ECS Fargate using Terraform, with an Application Load Balancer routing traffic to the search service. Load testing was performed using Locust with 20 concurrent users across four distinct test phases.

2. Architecture

The system consists of two microservices deployed as separate ECS Fargate tasks:

- Product Search Service (port 8080): Generates 100,000 products at startup using `sync.Map`, handles search queries by checking the first 100 products, and calls the downstream recommendation service for each request. Exposed behind an Application Load Balancer.
- Recommendation Service (port 8081): Returns product recommendations. Supports runtime failure injection via a `/mode` endpoint to simulate slow responses (3–8 second delay), errors (80% failure rate), or progressive degradation.

The search service has two operational modes controlled by the `RESILIENCE` environment variable, toggled via a single Terraform variable:

- `RESILIENCE=false`: No protection. The service waits indefinitely for downstream responses, making it vulnerable to cascading failures.
- `RESILIENCE=true`: Three resilience patterns activate — Fail Fast (500ms timeout), Circuit Breaker (opens after 5 failures, retries after 15s), and Bulkhead (max 10 concurrent downstream calls). When the downstream is unavailable, the service returns fallback recommendations.

3. Deployment

The entire infrastructure was provisioned using Terraform with a single `deploy-aws.sh` script that builds Docker images, pushes them to Amazon ECR, and deploys to ECS Fargate.

Apply complete! Resources: 4 added, 2 changed, 1 destroyed.

Outputs:

```
alb_dns = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com"
cluster_name = "crash-recovery-cluster"
metrics_url = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/metrics"
resilience_enabled = false
search_service_url = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/products/search?q=alpha"
```

=====

DEPLOYMENT COMPLETE!

=====

```
alb_dns = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com"
cluster_name = "crash-recovery-cluster"
metrics_url = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/metrics"
resilience_enabled = false
search_service_url = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/products/search?q=alpha"
```

Figure 1: Terraform deployment output showing successful creation of ALB, ECS cluster, services, and Service Discovery.

The screenshot displays the AWS ECS Console for the 'crash-recovery-cluster'. The cluster overview shows it is 'Active' with 2 running tasks. Below, the 'Services' tab lists two services: 'crash-recovery-recommendation' and 'crash-recovery-search', both running on FARGATE with 1/1 tasks running.

crash-recovery-cluster							
Cluster overview		Status	CloudWatch monitoring	Registered container instances			
ARN	arn:aws:ecs:us-west-2:591389222803:cluster/crash-recovery-cluster	Active	Default	-			
Services		Tasks					
Draining	-	Pending	-	Running			
	2			2			

Services (2)							
Service name	ARN	Status	Scheduling strategy	Launch type	Task definition	Deployments and tasks	Last deployment
crash-recovery-recommendation	arn:aws:ecs:us-west-2:591389222803:task-definition/crash-recovery-recommendation	Active	REPLICA	FARGATE	crash-reco...	1/1 Tasks running	Completed
crash-recovery-search	arn:aws:ecs:us-west-2:591389222803:task-definition/crash-recovery-search	Active	REPLICA	FARGATE	crash-reco...	1/1 Tasks running	In progress

Figure 2: AWS ECS Console showing crash-recovery-cluster with both services (search and recommendation) running on Fargate.

```
PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Midterm\Part II\crash-recovery> curl http://crash-recovery-alb-1591838
157.us-west-2.elb.amazonaws.com/health

StatusCode      : 200
StatusDescription : OK
Content         : ok
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 2
                  Content-Type: text/plain; charset=utf-8
                  Date: Sat, 28 Feb 2026 22:16:25 GMT

                  ok
Forms           : {}
Headers         : {[Connection, keep-alive], [Content-Length, 2], [Content-Type, text/plain; charset=utf-8], [Date,
                  Sat, 28 Feb 2026 22:16:25 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 2
```

Figure 3: Health check confirming the search service is reachable through the ALB (HTTP 200, content: "ok").

4. Phase 1: The Crash (No Resilience)

4.1 Baseline Test (Healthy Downstream)

With both services running normally (RESILIENCE=false, downstream in "none" mode), a 1-minute Locust test with 20 concurrent users established the baseline performance:

Metric	Value	Notes
Requests/sec (RPS)	89	Aggregate across all endpoints
Avg Response Time	16 ms	Fast: search + downstream call
Median Response Time	15 ms	Very consistent
Max Response Time	151 ms	Occasional tail latency
Total Requests	6,854	In 1 minute
Failures	0	100% success rate

4.2 Triggering the Failure

The recommendation service was switched to "slow" mode via its runtime API, causing it to inject a random 3–8 second delay on every response:

```
(.venv) PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Midterm\Part II\crash-recovery> curl "http://35.90.56.98:8081/mode?mode=slow"

StatusCode      : 200
StatusDescription : OK
Content         : {"new_mode":"slow","old_mode":"slow","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 52
                  Content-Type: application/json
                  Date: Sat, 28 Feb 2026 23:10:19 GMT

                  {"new_mode":"slow","old_mode":"slow","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 52], [Content-Type, application/json], [Date, Sat, 28 Feb 2026 23:10:19 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 52
```

Figure 4: Triggering slow mode on the recommendation service (old_mode: "none" → new_mode: "slow").

4.3 Load Test Under Failure

With the downstream now responding in 3–8 seconds, the same 20-user Locust test was run for 2 minutes. The results were catastrophic:

Metric	Baseline	Under Failure	Change
RPS	89	4	–95%
Avg Latency	16 ms	4,644 ms	290x slower
Median Latency	15 ms	5,200 ms	347x slower
Max Latency	151 ms	8,015 ms	53x slower
Total Requests	6,854	478	–93%

The cascading failure mechanism: every incoming request spawns a goroutine that blocks waiting for the slow downstream. With the downstream taking 3–8 seconds per response, goroutines pile up, and the system can only process approximately 4 requests per second instead of 89. Throughput dropped by 95%.

4.4 Metrics and CloudWatch

```
(.venv) PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Midterm\Part II\crash-recovery> curl "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/metrics"

StatusCode      : 200
StatusDescription : OK
Content         : {"failed_rec":0,"fallback_rec":0,"resilience_enabled":false,"successful_rec":6587,"timeout_rec":0,"total_searches":6587}
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 125
                  Content-Type: application/json
                  Date: Sat, 28 Feb 2026 23:16:51 GMT

                  {"failed_rec":0,"fallback_rec":0,"resilience_enabled":false,"succ...
Forms           : {}
Headers         : {[Connection, keep-alive], [Content-Length, 125], [Content-Type, application/json], [Date, Sat, 28 Feb 2026 23:16:51 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 125
```

Figure 5: Service metrics with `resilience_enabled: false`. All 6,587 requests "succeeded" — they just took 3–8 seconds each. No fallback or timeout mechanism exists.

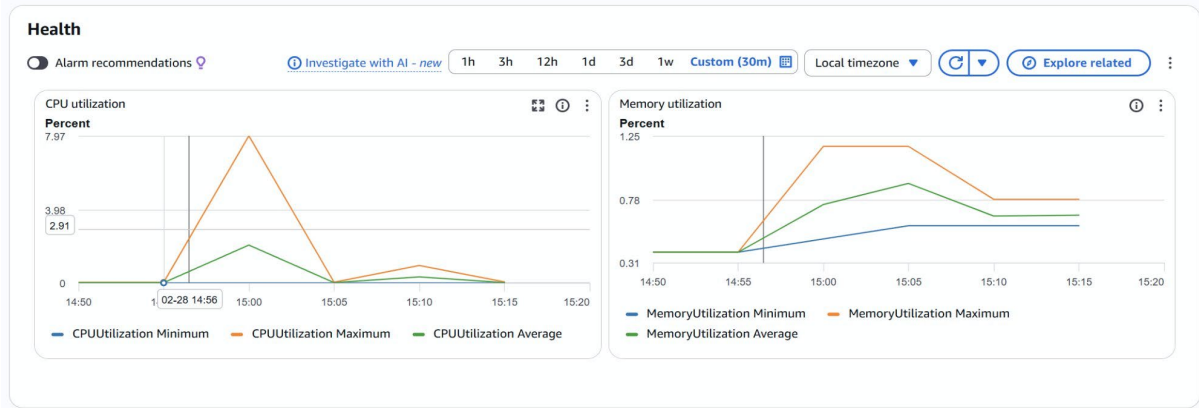


Figure 6: CloudWatch CPU and Memory. CPU stayed below 8% — the bottleneck was I/O wait (goroutines blocked on network), not computation. Memory rose as blocked goroutines accumulated.

5. Phase 2: The Fix (With Resilience)

5.1 Enabling Resilience Patterns

The fix was deployed by changing `resilience_enabled` from `false` to `true` in `terraform.tfvars` and running `terraform apply`. This updated the search service's RESILIENCE environment variable, triggering a new ECS task deployment:

```

Plan: 1 to add, 1 to change, 1 to destroy.

Changes to Outputs:
  ~ resilience_enabled = false -> true
aws_ecs_task_definition.search: Destroying... [id=crash-recovery-search]
aws_ecs_task_definition.search: Destruction complete after 0s
aws_ecs_task_definition.search: Creating...
aws_ecs_task_definition.search: Creation complete after 0s [id=crash-recovery-search]
aws_ecs_service.search: Modifying... [id=arn:aws:ecs:us-west-2:591389222803:service/crash-recovery-cluster/crash-recovery-search]
aws_ecs_service.search: Modifications complete after 1s [id=arn:aws:ecs:us-west-2:591389222803:service/crash-recovery-cluster/crash-recovery-search]

Apply complete! Resources: 1 added, 1 changed, 1 destroyed.

Outputs:
alb_dns = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com"
cluster_name = "crash-recovery-cluster"
metrics_url = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/metrics"
resilience_enabled = true
search_service_url = "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/products/search?q=alpha"

```

Figure 7: Terraform apply showing `resilience_enabled = false → true`. Task definition recreated, service updated in 1 second.

5.2 Three Resilience Patterns

Pattern 1: Fail Fast (Timeout)

The HTTP client for downstream calls uses a 500ms timeout. Instead of waiting 3–8 seconds, the service times out after half a second and returns fallback recommendations. This prevents goroutines from blocking indefinitely.

Pattern 2: Circuit Breaker

After 5 consecutive failures (timeouts), the circuit breaker opens and stops making downstream calls entirely for 15 seconds. All requests immediately receive fallback recommendations with zero network overhead. After the cooldown, it enters "half-open" state and allows test requests through. If they succeed, the circuit closes and normal operation resumes.

Pattern 3: Bulkhead

A semaphore limits concurrent downstream calls to 10. Even before the circuit breaker trips, only 10 goroutines can wait on the downstream simultaneously. Remaining requests are served with fallback recommendations immediately, isolating the failure.

5.3 Confirming Failure Conditions

Before testing, the downstream was confirmed to still be in slow mode:

```
(.venv) PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Midterm\Part II\crash-recovery> curl "http://35.90.56.98:8081,
ode?mode=slow"

StatusCode      : 200
StatusDescription : OK
Content         : {"new_mode":"slow","old_mode":"slow","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 52
                  Content-Type: application/json
                  Date: Sat, 28 Feb 2026 23:28:42 GMT

                  {"new_mode":"slow","old_mode":"slow","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 52], [Content-Type, application/json], [Date, Sat, 28 Feb 2026 23:28:42 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 52
```

Figure 8: Confirming the recommendation service is still in slow mode (same failure conditions as Phase 1).

5.4 Load Test Results

With identical failure conditions (downstream in slow mode), the 20-user Locust test was repeated for 2 minutes:

Metric	Baseline	Crashed	Resilient	Recovery
RPS	89	4	89	100%
Avg Latency	16 ms	4,644 ms	18 ms	99.6%
Median	15 ms	5,200 ms	14 ms	99.7%
Max	151 ms	8,015 ms	538 ms	93.3%
Requests	6,854	478	10,683	—

The resilience patterns restored throughput to baseline levels (89 RPS) with negligible latency increase (18ms vs 16ms). The max latency of 538ms represents the few requests that hit the 500ms timeout before the circuit breaker opened. Once open, all subsequent requests were served instantly with fallback data.

5.5 Resilience Metrics

```
(.venv) PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Midterm\Part II\crash-recovery> curl "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/metrics"

StatusCode      : 200
StatusDescription : OK
Content         : {"bulkhead":{"max_concurrent":10,"total_admitted":75,"total_rejected":8},"circuit_breaker":{"state":"open","total_failures":83,"total_fallbacks":8809,"total_opens":8,"total_requests":8892},"failed_rec...
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 313
                  Content-Type: application/json
                  Date: Sat, 28 Feb 2026 23:34:23 GMT

                  {"bulkhead":{"max_concurrent":10,"total_admitted":75,"total_rejecte...
Forms           : {}
Headers        : {[Connection, keep-alive], [Content-Length, 313], [Content-Type, application/json], [Date, Sat, 28 Feb 2026 23:34:23 GMT]}
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     : mshhtml.HTMLDocumentClass
RawContentLength : 313
```

Figure 9: Resilience metrics in action. Circuit breaker: "open" (protecting the system). 8,809 fallback responses served. 83 timeout failures before breaker tripped. Bulkhead rejected 8 requests exceeding the 10-concurrent limit.

6. Phase 3: Automatic Recovery

6.1 Restoring the Downstream

The recommendation service was switched back to healthy mode:

```
(.venv) PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Midterm\Part II\crash-recovery> curl "http://35.90.56.98:8081/mode?mode=none"

StatusCode      : 200
StatusDescription : OK
Content         : {"new_mode":"none","old_mode":"slow","status":"ok"}

RawContent      : HTTP/1.1 200 OK
                  Content-Length: 52
                  Content-Type: application/json
                  Date: Sat, 28 Feb 2026 23:36:43 GMT

                  {"new_mode":"none","old_mode":"slow","status":"ok"}

Forms           : {}
Headers         : {[Content-Length, 52], [Content-Type, application/json], [Date, Sat, 28 Feb 2026 23:36:43 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 52
```

Figure 10: Switching the recommendation service back to normal (*old_mode*: "slow" → *new_mode*: "none").

6.2 Circuit Breaker Recovery

After restoring the downstream, a final 2-minute Locust test was run. The circuit breaker automatically recovered through its state machine: OPEN → HALF_OPEN (after 15-second timeout) → CLOSED (after 3 successful probe requests).

Metric	Baseline	Recovered	Difference
RPS	89	89	Identical
Avg Latency	16 ms	17 ms	+1 ms
Median	15 ms	17 ms	+2 ms
Max	151 ms	199 ms	Normal range
Requests	6,854	10,648	Fully recovered

```
(.venv) PS C:\Users\98999\Desktop\CS6650-Distributed-Systems\Midterm\Part II\crash-recovery> curl "http://crash-recovery-alb-1591838157.us-west-2.elb.amazonaws.com/metrics"

StatusCode      : 200
StatusDescription : OK
Content         : {"bulkhead":{"max_concurrent":10,"total_admitted":7888,"total_rejected":13},"circuit_breaker":{"state":
                  "closed","total_failures":88,"total_fallbacks":9957,"total_opens":9,"total_requests":17858},"fail...
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Content-Length: 323
                  Content-Type: application/json
                  Date: Sat, 28 Feb 2026 23:47:28 GMT

                  {"bulkhead":{"max_concurrent":10,"total_admitted":7888,"total_rejec...
Forms           : {}
Headers         : [[Connection, keep-alive], [Content-Length, 323], [Content-Type, application/json], [Date, Sat, 28 Feb
                  2026 23:47:28 GMT]]
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 323
```

Figure 11: Final metrics showing circuit_breaker state: "closed". The system automatically detected recovery and resumed live downstream calls. 17,858 total requests processed across all test phases.

7. Key Takeaways

1. Cascading failures are silent killers.

The unprotected system didn't return errors — it returned successful responses, just 290x slower. From a monitoring perspective, error rates looked "fine" while throughput dropped 95%. Without resilience patterns, a single slow dependency can silently bring down an entire service.

2. CPU is not always the bottleneck.

CloudWatch showed CPU utilization below 8% during the cascading failure. The bottleneck was I/O wait — goroutines blocked on network calls. This is a common trap: teams scale up CPU or add instances when the real problem is unbounded downstream call duration.

3. Defense in depth works.

The three patterns complement each other: Fail Fast limits individual request damage (500ms cap), Circuit Breaker eliminates network overhead entirely after detecting failure (0ms for cached fallback), and Bulkhead prevents resource exhaustion even before the breaker trips (max 10 blocked goroutines). Together, they maintained 100% throughput recovery under identical failure conditions.

4. Graceful degradation beats hard failure.

With resilience enabled, users received fallback recommendations instead of errors or multi-second waits. The search functionality continued working perfectly — only the recommendation quality degraded temporarily. This is a key principle from Sam Newman: it's better to return slightly stale or generic data than to fail entirely.

5. Automatic recovery eliminates manual intervention.

The circuit breaker's state machine (CLOSED → OPEN → HALF_OPEN → CLOSED) meant the system recovered automatically once the downstream was restored. No restarts, no redeployments, no manual intervention — the service detected recovery and resumed normal operation within 15 seconds.

8. Tools and Technologies

Component	Technology
Search Service	Go (net/http, sync.Map, custom circuit breaker/bulkhead)
Recommendation Service	Go (net/http, runtime failure injection via /mode endpoint)
Containerization	Docker (multi-stage build, Alpine base image)
Infrastructure	Terraform (ECS Fargate, ALB, Target Group, Service Discovery, CloudWatch Logs)
Container Registry	Amazon ECR
Load Testing	Locust (HttpUser, 20 concurrent users, 4 test phases)
Monitoring	CloudWatch (CPU/Memory), custom /metrics endpoint (circuit breaker state, bulkhead stats)